

8.2 배운 내용을 정리

오늘 목표는 리버싱용 게임에서 지난번에 한 핵 개발의 마지막 단계인 벽까지 그려서 완벽한 핵을 만드는 것이다. 이를 위해서 ida를 열고 먼저 벽돌을 그리는 부분을 찾기 위해서 `sdl_fillColor`, `SDL_RenderFillRect_0`와 같이 그리는 함수들을 찾아봤더니

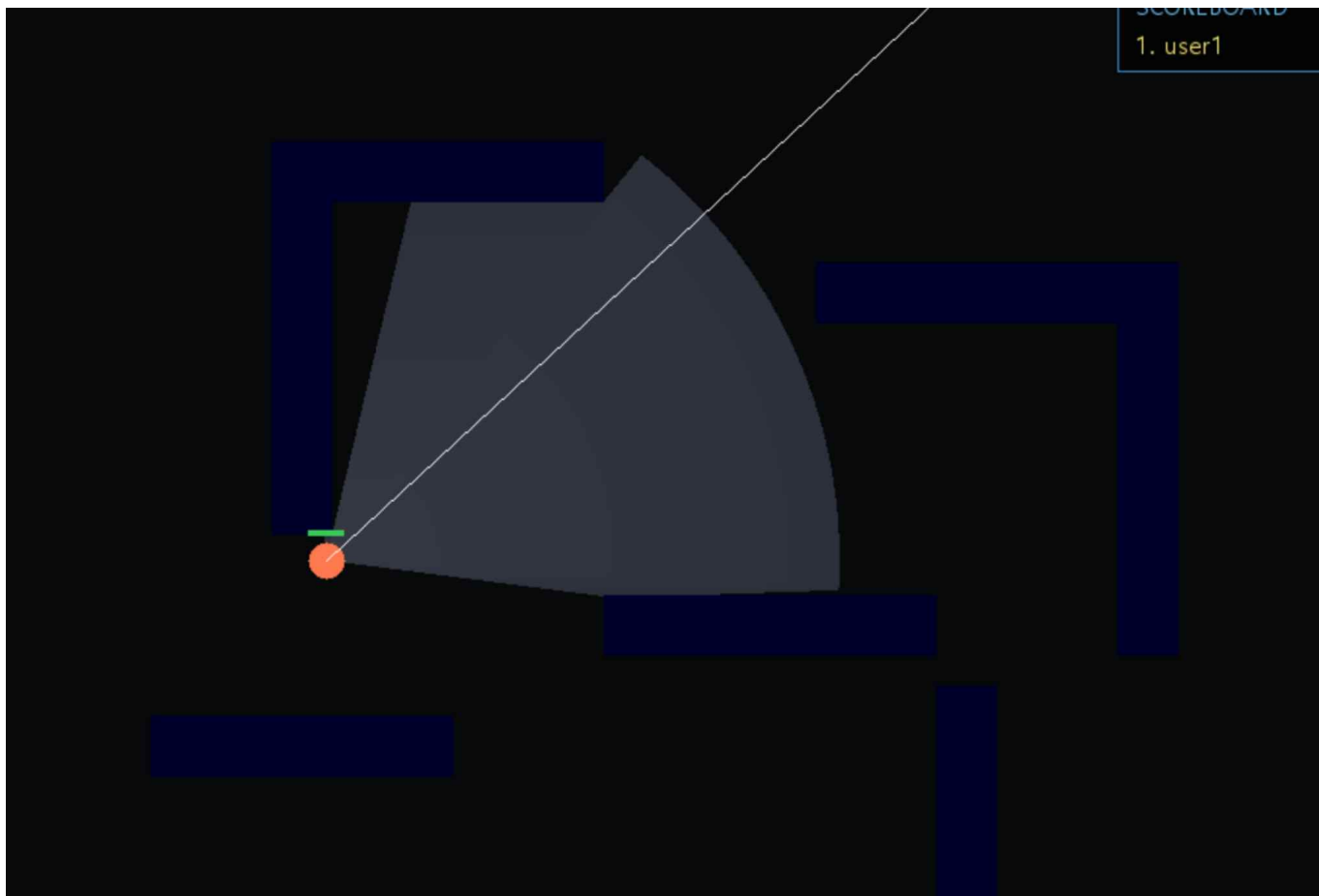
```
11 {
12     SDL_SetRenderDrawColor_0(a1, 50, 52, 60, 255);
13     SDL_RenderDrawRect_0();
14     SDL_SetRenderDrawColor_0(a1, 28, 30, 38, 255);
15     v12 = a2 + 25;
16     v11 = 321;
17     v13 = 0x22000000166LL;
18     SDL_RenderFillRect_0(a1, (__int64)&v11);
19     goto LABEL_6;
20 }
21 SDL_SetRenderDrawColor_0(a1, 90, 110, 150, 255);
22 SDL_RenderDrawRect_0();
23 SDL_SetRenderDrawColor_0(a1, 28, 30, 38, 255);
24 v12 = a2 + 25;
25 v11 = 321;
26 v13 = 0x22000000166LL;
```

이런식으로 벽돌 칠하는 정보를 넘기는 듯한 함수가 있었고 이들 중 어떤 게 벽돌을 가리키는 지 확인하기 위해서 벽돌의 색깔이 어두운 회색이라는 정보를 통해 후보를 추려내고 xdbg에서 그 지점을 중단점을 설정하고 실행해보니 벽에 부딪치는 경우나 움직일 때에는 추가적인 정보를 주지 않고 처음 이 프로그램을 실행할 때 한 번만 중단점이 걸리는 걸 보고 처음 초기화 될때만 벽정보가 저장된다는 사실을 깨달았다.

RIP	00007FF74AD02A51	E8 822C0000	call <JMP.&SDL_SetRenderDrawColor>	RAX	0000000000000000
	00007FF74AD02A56	44:8B8424 AC000000	mov r8d,dword ptr ss:[rsp+AC]	RBX	0000000B4CDFBD90
	00007FF74AD02A5E	45:85C0	test r8d,r8d	RCX	000001DF32D287A0
	00007FF74AD02A61	7E 42	jle client.7FF74AD02AA5	RDY	000000000000000F
	00007FF74AD02A63	48:8B4424 60	mov rax,qword ptr ss:[rsp+60]	RBP	FFFFFFFFFFFFFFFF
	00007FF74AD02A68	48:8B4C24 70	mov rcx,qword ptr ss:[rsp+70]	RSP	0000000B4CDE98E0
	00007FF74AD02A6D	66:0FEFF6	pxor xmm6,xmm6		
	00007FF74AD02A71	4C:8B6424 68	mov r12,qword ptr ss:[rsp+68]		

여기 지점에서 벽이 맞는지 더 자세히 확인하기 위해서 벽의 색깔을 기존의 어두운 검정색에서 파란색으로 rgb값의 인자를 바꿔 보았다.

00007FF74AD02A35	C/4424 20 FF000000	mov dword ptr ss:[rsp+20],FF
00007FF74AD02A3D	41:B8 00000000	mov r8d,0
00007FF74AD02A43	4C:89F1	mov rcx,r14
00007FF74AD02A46	41:B9 FF000000	mov r9d,FF
00007FF74AD02A4C	BA 00000000	mov edx,0
00007FF74AD02A51	E8 822C0000	call <JMP.&SDL_SetRenderDrawColor>
00007FF74AD02A56	44:8B8424 AC000000	mov r8d,dword ptr ss:[rsp+AC]
00007FF74AD02A5E	45:85C0	test r8d,r8d
00007FF74AD02A61	7E 42	jle client.7FF74AD02AA5
00007FF74AD02A63	48:8B4424 60	mov rax,qword ptr ss:[rsp+60]
00007FF74AD02A68	48:8B4C24 70	mov rcx,qword ptr ss:[rsp+70]



이렇게 벽이 파란색으로 나오는 걸 확인했다 그러면 이제 이 벽을 실제 그리는 fillrect를 아래에서 찾아보면

```
SDL_SetRenderTarget(Renderer_0, 0);
SDL_SetRenderDrawColor_0(Renderer_0, 55, 58, 70, 255); // 벽돌을 그리는 부분으로 추정되는 부분
SDL_RenderClear(Renderer_0);
SDL_SetRenderDrawColor_0(Renderer_0, 95, 100, 115, 255); // 실제 벽 부분이다
if ( v258 > 0 )
{
    ;L_95:
    v90 = (const __m128i *)v294;
    do
    {
        v91 = _mm_loadl_epi64(v90);
        v90 = (const __m128i *)((char *)v90 + 8);
        *(__m128i *)Destination = _mm_unpacklo_epi16(v91, (__m128i)0LL);
        SDL_RenderFillRect_0(Renderer_0, (__int64)Destination);
    }
    while ( &v294[Size] != (_BYTE *)v90 );
}
```

이렇게 두 번째 인자를 destination으로 위치와 크기 정보를 보내는 데 이 때 이 dst는 v90에서 unpack하는 거고 이는 v294에서 값을 가져온다는 소리다. v294를 xref를 통해

서 확인하면

```
530 memcpy_0(v294, &g_walls, Size);
531 LeaveCriticalSection(&g_lock);
532 if ( v44 == -1 )
533     goto LABEL_236;
534 v53 = 0;
```

이렇게 walls정보를 그대로 가져온다는 걸 확인할 수 있고 이를 16바이트씩 가져와서 최적화를 unpacek을 통해 해서 8바이트를 전달하는 형식이다.

그 다음에는 sdl_rect정보를 바탕으로 destination을 분석하기 위해 구조체를 만들어 보면 shift + f1 누르면 구조체가 나온다 ida는 기본적으로 구조체를 만들지 않음으로 이런 구조체가 있으면 이해하기 쉬운 프로그램에서는 내가 직접 만들면 더 이해하기가 쉬워질 수 있다 우클릭 후 add type을 누르고 c언어 문법에 맞게 구조체를 작성하면

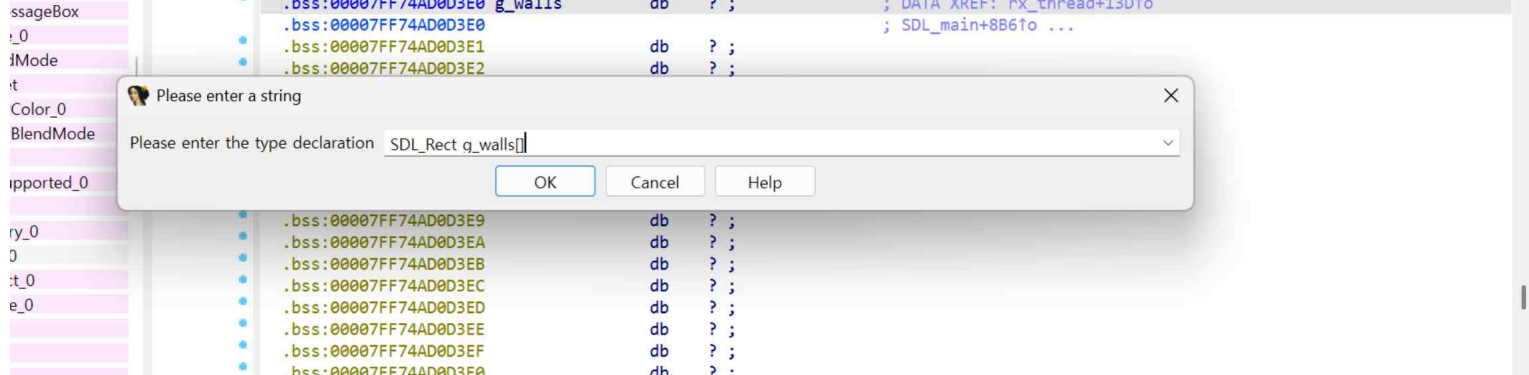
```
00000010 char szDescription[257];
00000111 char szSystemStatus[129];
00000192 // padding byte
00000193 // padding byte
00000194 // padding byte
00000195 // padding byte
00000196 // padding byte
00000197 // padding byte
00000198 };

00000008 typedef DWORD *LPDWORD; // XREF: SDL_ma:

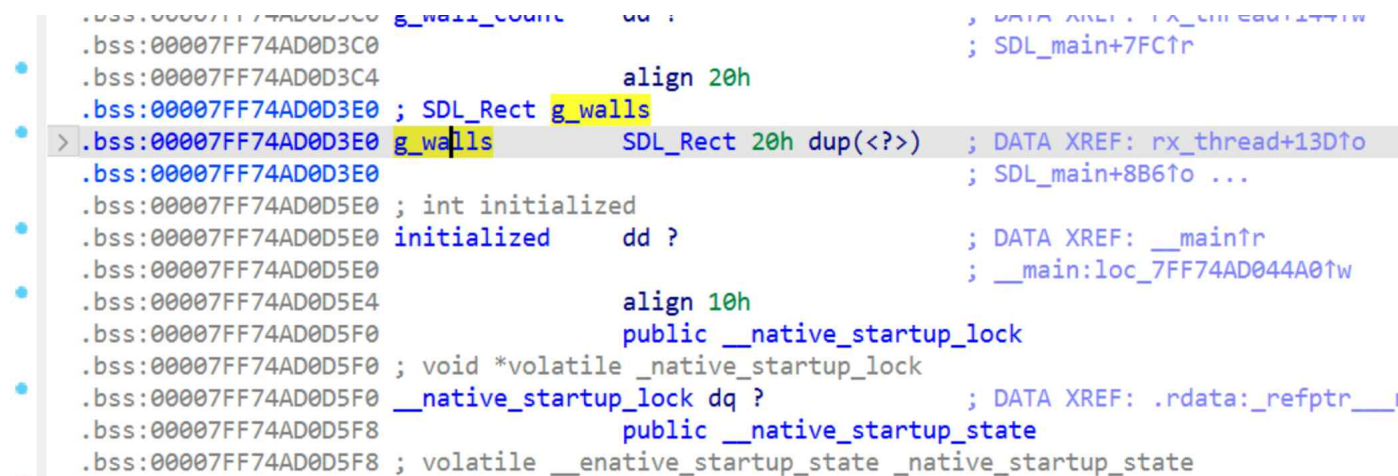
00000000 struct SDL_Rect // sizeof=0x10
00000000 {
00000000     int x;
00000004     int y;
00000008     int width;
0000000C     int height;
00000010 };

line 109 of 144, /dwarf/SDL_ #143:SDL_Rect
```

위와 같이 구조체가 생긴다. 이번 수업에서는 sdl_rect가 저런 구조체를 가진다는 정보를 이용하기 위해 sdl_rect구조체를 만들었다. 위에 나오는 destination은 이 type의 구조체를 가리키는 포인터다. 따라서 이를 통해 우리는 g_walls 가 sdl_rect에 저장되는 배열이라는 사실을 추론할 수 있다. 따라서 g_walls의 타입을 그렇게 바꿔준다.



그렇게 한 다음에는 개수를 정확히 모를 때는 `SDL_Rect g_walls`처럼 구조체 형태로 저장하고 우클릭 후 array를 누르면 ida가 배열 크기를 추정해서 알려준다.



또한 그렇게 하고 이 부분은 bss section이라 프로세스가 되지 않은 상태에서는 ?만 뜨는 공간이다 실제 어떤 데이터가 있는지 보려면 xdbg64에 `g_walls`의 덤프창에서 찾아가면 알 수 있다 왜냐하면 xdbg64에서는 실제로 프로세스가 동작하고 있기 때문이다.

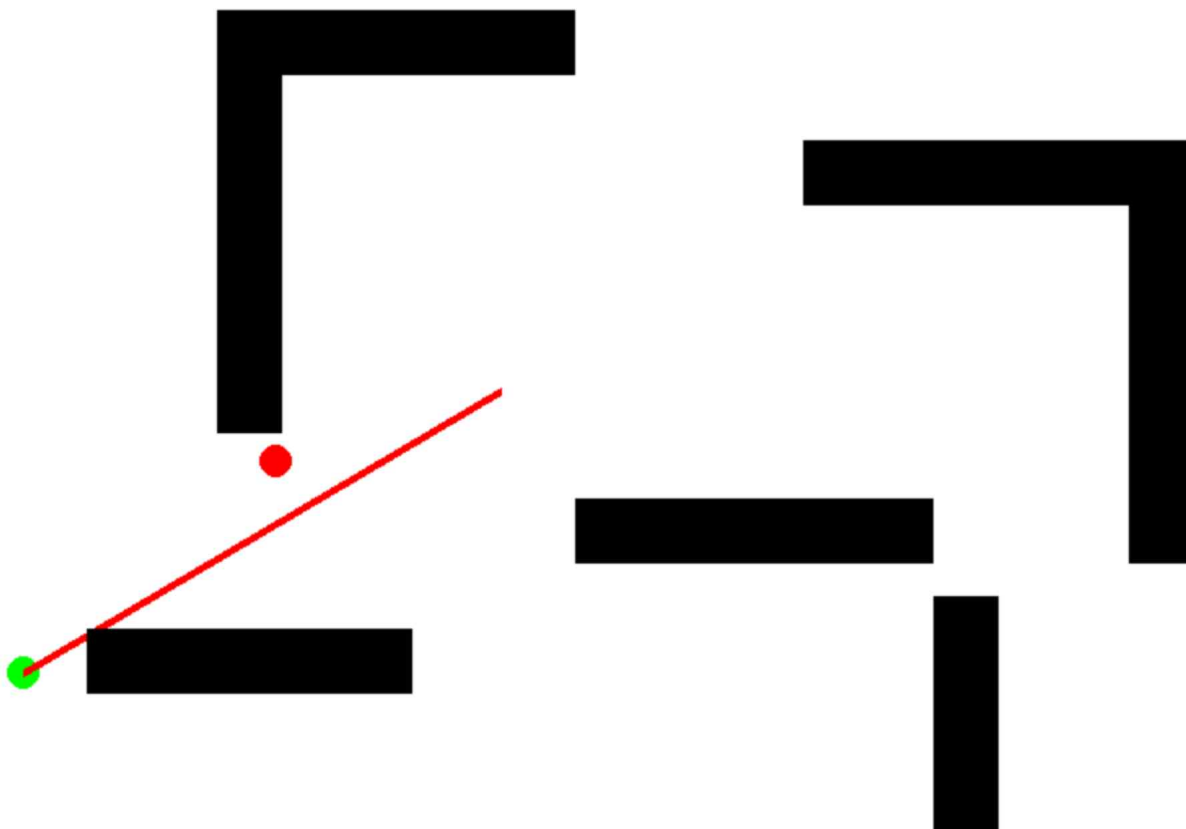
주소	Hex	ASCII
00007FF74AD0D3	C8 00 78 00 28 00 04 01 C8 00 78 00 DC 00 28 00	È.x.(...È.x.Ü.(.
00007FF74AD0D3	30 02 C8 00 F0 00 28 00 F8 02 C8 00 28 00 04 01	0.È.ð.(.ø.È.(...
00007FF74AD0D4	A4 01 A4 01 DC 00 28 00 78 00 F4 01 C8 00 28 00	æ.æ.Ü.(.x.ô.È.(.
00007FF74AD0D4	80 02 E0 01 28 00 B4 00 00 00 00 00 00 00 00	..à.(.....
00007FF74AD0D4	00 00 00 00 00 00 00 00 00 00 00 00 00 00	

가면 벽 데이터 배열을 볼 수 있고 이 자료를 분석하면 약간 2바이트씩 가져오는 거 같았다 이를 통해서 `sdl-rect`는 16바이트로 정보를 관리하는 데 실제 프로그램에서는 2바이트 * 4 = 8바이트로 unpack해 관리하는 거 같았다.)각각 `x,y,width,height`정보를 주는 거다.

이제 이 가설을 확인하기 위해서 파이썬으로 가서 packet id가 2인 경우에 대해서 벽 정보를 받아서 일단 출력을 해 보았더니 신기하게 xdbg에서 중단점을 걸어도 멈추지 않아서 아마 처음에 receive를 한 거 같다는 의심이 들었다. 그래서 처음 receive한 packet을 보니 벽 정보와 유저의 번호를 전달하는 패킷이었던 것이었다.

```
IDA View-A  Pseudocode-A  Hex View-1  Local Types  Im
;8      v10 = Buffer[offset + 11];
;9      v11 = (unsigned __int8)*v9;
;10     EnterCriticalSection(&g_lock);
;11     v12 = v10;
;12     if ( v10 > 0x40u )
;13     {
;14         v12 = 64;
;15         v13 = 512;
;16     }
;17     else
;18     {
;19         v13 = 8LL * v10;
;20     }
;21     data_start = v11;
;22     g_wall_count = v12;
;23     if ( (unsigned int)v13 >= 8 )
;24     {
;25         memcpy(&g_walls, &Buffer[offset + 12], 8 * (v13 >> 3));
;26         ((void (__fastcall *)(_RTL_CRITICAL_SECTION *))LeaveCriticalSection)(&g_lock);
;27     }
;28     else
;29     {
;30         if ( packet_id == 5 )                // 플레이어 좌표 수신
```

그리고 이 부분을 통해서 v10은 벽의 개수이고 12번째 요소부터 벽의 정보를 전달한다는 걸 알 수 있었다. 따라서 12부터 2바이트 단위로 파싱해서 출력했더니 예상대로 벽의 x,y,width, height정보가 다 나왔다. 이 정보들을 이제 취합해서 tuple로 만들어서 rect로 벽을 pygame으로 그리면



이렇게 완전한 핵이 만들어졌다.

